Homework 4: Spatial and Temporal Blocking for Model Evaluation

GEOL 599: Machine Learning

```
# If needed, install these packages first:
# install.packages(c("tidyverse", "tidymodels", "terra", "ranger", "gstat", "sf", "sp"))

library(tidyverse)
library(tidymodels)
library(terra)
library(ranger)
library(gstat)
library(sf)
library(sp)

tidymodels_prefer()
set.seed(42)
```

Goal

Compare ordinary random cross-validation with blocked cross-validation for spatially and temporally correlated environmental data.

In this homework, you will use a gridded **sea surface temperature (SST)** dataset. SST is a good example because nearby ocean grid cells tend to have similar temperatures, and nearby months or years are also related. This means ordinary random cross-validation can give an overly optimistic estimate of model performance.

Sections marked **TODO** are places where you should write or modify code.

1. Load the Sea Surface Temperature Dataset

The data are stored as a NetCDF file, a common format for gridded environmental and oceanographic data. Instead of rows and columns only, the data have dimensions such as `time`, `lat`, and `lon`.

```
data_url <- "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/sst.nc"
file_path <- "sst.nc"

if (!file.exists(file_path)) {
  download.file(data_url, destfile = file_path, mode = "wb")
}

sst_rast <- rast(file_path)
sst_rast
```

```
## class       : SpatRaster
## size        : 31, 43, 116  (nrow, ncol, nlyr)
## resolution  : 8.52381, 5.866667  (x, y)
## extent      : -4.261905, 362.2619, -90.93333, 90.93333  (xmin, xmax, ymin, ymax)
## coord. ref. :
```

```
## source      : sst.nc
## names       : sst_1, sst_2, sst_3, sst_4, sst_5, sst_6,  ...
## unit        : degC
## time (days) : 1992-01-01 to 2020-10-01 (116 steps)
```

The SST variable is stored as a multilayer raster. Each layer represents one time step.

```
nlyr(sst_rast)
```

```
## [1] 116
```

```
ext(sst_rast)
```

```
## SpatExtent : -4.26190476190476, 362.261904761905, -90.9333333333333, 90.9333333333333 (xmin, xmax, y
```

```
res(sst_rast)
```

```
## [1] 8.523810 5.866667
```

```
time(sst_rast)
```

```
##   [1] "1992-01-01" "1992-04-01" "1992-07-01" "1992-10-01" "1993-01-01"
##   [6] "1993-04-01" "1993-07-01" "1993-10-01" "1994-01-01" "1994-04-01"
##  [11] "1994-07-01" "1994-10-01" "1995-01-01" "1995-04-01" "1995-07-01"
##  [16] "1995-10-01" "1996-01-01" "1996-04-01" "1996-07-01" "1996-10-01"
##  [21] "1997-01-01" "1997-04-01" "1997-07-01" "1997-10-01" "1998-01-01"
##  [26] "1998-04-01" "1998-07-01" "1998-10-01" "1999-01-01" "1999-04-01"
##  [31] "1999-07-01" "1999-10-01" "2000-01-01" "2000-04-01" "2000-07-01"
##  [36] "2000-10-01" "2001-01-01" "2001-04-01" "2001-07-01" "2001-10-01"
##  [41] "2002-01-01" "2002-04-01" "2002-07-01" "2002-10-01" "2003-01-01"
##  [46] "2003-04-01" "2003-07-01" "2003-10-01" "2004-01-01" "2004-04-01"
##  [51] "2004-07-01" "2004-10-01" "2005-01-01" "2005-04-01" "2005-07-01"
##  [56] "2005-10-01" "2006-01-01" "2006-04-01" "2006-07-01" "2006-10-01"
##  [61] "2007-01-01" "2007-04-01" "2007-07-01" "2007-10-01" "2008-01-01"
##  [66] "2008-04-01" "2008-07-01" "2008-10-01" "2009-01-01" "2009-04-01"
##  [71] "2009-07-01" "2009-10-01" "2010-01-01" "2010-04-01" "2010-07-01"
##  [76] "2010-10-01" "2011-01-01" "2011-04-01" "2011-07-01" "2011-10-01"
##  [81] "2012-01-01" "2012-04-01" "2012-07-01" "2012-10-01" "2013-01-01"
##  [86] "2013-04-01" "2013-07-01" "2013-10-01" "2014-01-01" "2014-04-01"
##  [91] "2014-07-01" "2014-10-01" "2015-01-01" "2015-04-01" "2015-07-01"
##  [96] "2015-10-01" "2016-01-01" "2016-04-01" "2016-07-01" "2016-10-01"
## [101] "2017-01-01" "2017-04-01" "2017-07-01" "2017-10-01" "2018-01-01"
## [106] "2018-04-01" "2018-07-01" "2018-10-01" "2019-01-01" "2019-04-01"
## [111] "2019-07-01" "2019-10-01" "2020-01-01" "2020-04-01" "2020-07-01"
## [116] "2020-10-01"
```

Discussion

Look at the dataset metadata above.

Questions to answer:

1. What are the dimensions of the dataset?
2. What is the spacing between latitude and longitude grid cells?
3. How many time steps are included?
4. Why might neighboring grid cells have similar SST values?

There are 116 layers, with 31 rows and 43 columns in each layer.The resolution is 8.52381 longitudinal spacing and 5.866667 for latitude. There are 116 time steps from Jan 1, 1992 to

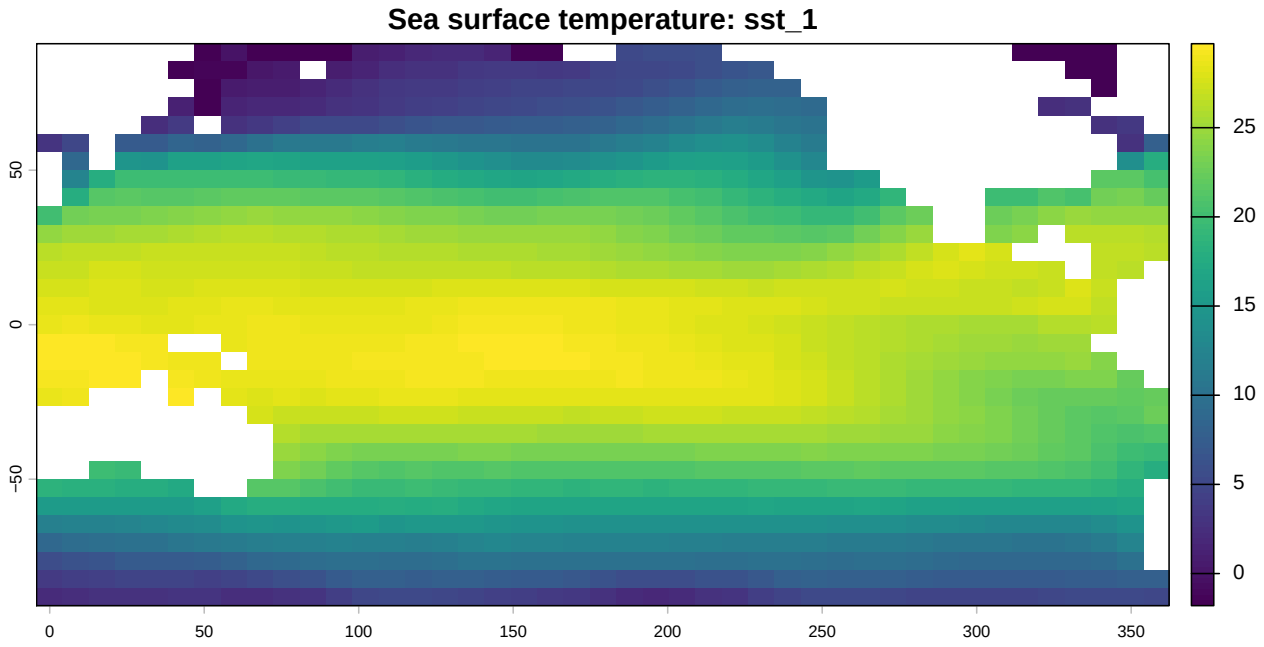
October 1, 2020. Neighboring grid cells would have similar SST values when the water is well mixed. I would only expect them to be different under conditions for a front or eddy.

2. Explore the Data Visually

Before modeling, inspect the target variable. SST has strong geographic structure and seasonal/temporal structure. The first plot shows the SST field for one time step.

```
one_time <- sst_rast[[1]]

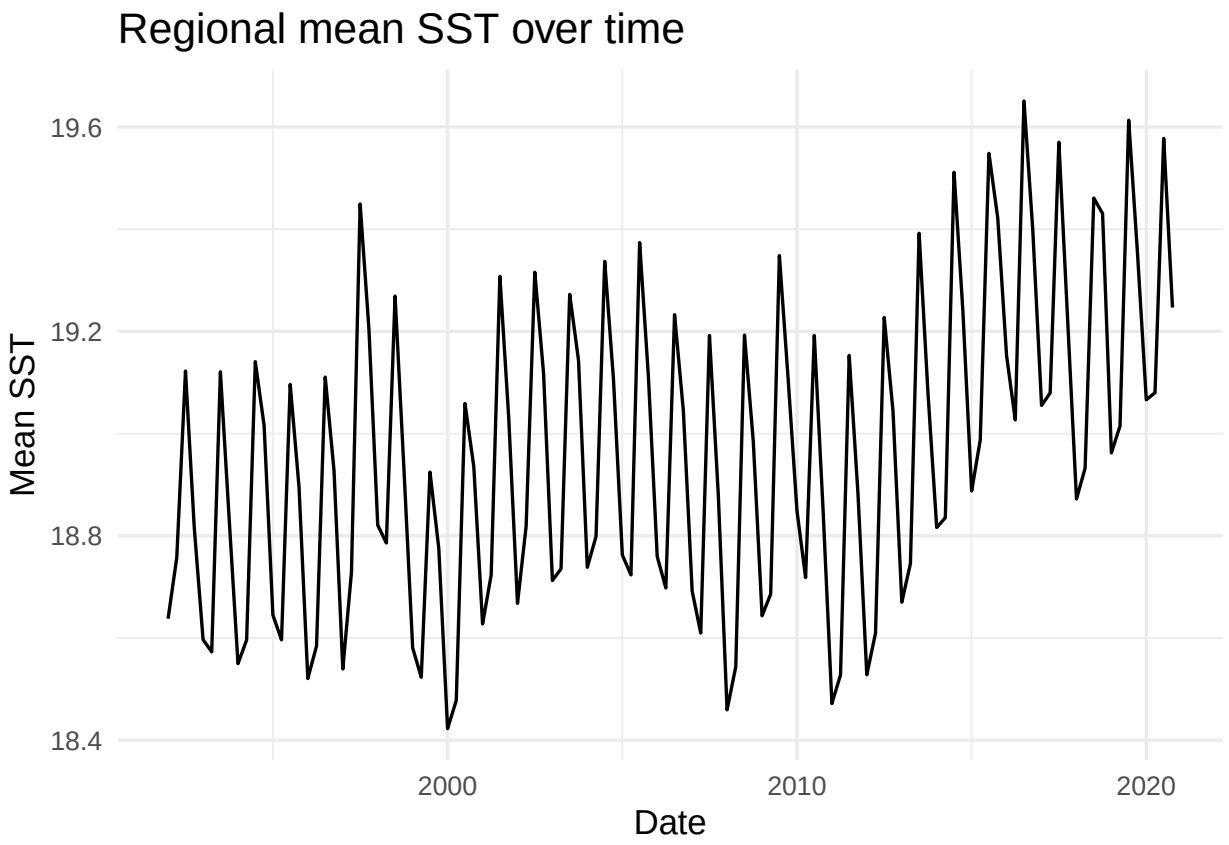
plot(one_time, main = paste("Sea surface temperature:", names(one_time)))
```



The next plot averages SST across the selected region for each time step.

```
regional_mean <- global(sst_rast, fun = "mean", na.rm = TRUE) %>%
  as_tibble() %>%
  mutate(
    layer = row_number(),
    date = time(sst_rast)
  )

ggplot(regional_mean, aes(x = date, y = mean)) +
  geom_line() +
  labs(
    title = "Regional mean SST over time",
    x = "Date",
    y = "Mean SST"
  ) +
  theme_minimal(base_size = 13)
```



TODO: Describe the Correlation Structure

Write a short answer after running the plots.

Prompt: Based on the maps and time series, where do you expect random cross-validation to be misleading? Consider both nearby locations and nearby times.

Your answer:There are spatial and seasonal trends that would make random cross validation misleading. SST along the equator is similar and SST in the same months will be similar for the years 1990-2010. TODO

3. Convert Gridded Data to a Machine-Learning Table

Most `tidymodels` workflows expect a table where each row is one observation. Here, one row will represent one grid cell at one time. The target is `sst`. The predictors will be simple features based on location, season, and time.

```
# Convert raster layers to a wide table with x/y columns and one column per time step.
sst_wide <- as.data.frame(sst_rast, xy = TRUE, na.rm = FALSE) %>%
  as_tibble() %>%
  rename(lon = x, lat = y)

# Use raster time metadata when available.
sst_dates <- time(sst_rast)

# Pivot from wide raster format to long machine-learning format.
df <- sst_wide %>%
  pivot_longer(
```

```
    cols = -c(lon, lat),
    names_to = "layer",
    values_to = "sst"
  ) %>%
  drop_na(sst) %>%
  mutate(
    layer_index = as.integer(factor(layer, levels = names(sst_rast))),
    time = sst_dates[layer_index],
    year = lubridate::year(time),
    month = lubridate::month(time),
    time_index = (year - min(year, na.rm = TRUE)) * 12 + (month - 1),
    month_sin = sin(2 * pi * month / 12),
    month_cos = cos(2 * pi * month / 12),
    lon_sin = sin(2 * pi * lon / 360),
    lon_cos = cos(2 * pi * lon / 360)
  )

dim(df)
```

```
## [1] 131544     13
```

```
head(df)
```

```
## # A tibble: 6 x 13
##     lon   lat layer   sst layer_index time        year month time_index
##   <dbl> <dbl> <chr> <dbl>       <int> <date>     <dbl> <dbl>      <dbl>
## 1  51.1    88 sst_1 -1.80           1 1992-01-01  1992     1          0
## 2  51.1    88 sst_2 -1.78           2 1992-04-01  1992     4          3
## 3  51.1    88 sst_3 10.0            3 1992-07-01  1992     7          6
## 4  51.1    88 sst_4  5.82           4 1992-10-01  1992    10          9
## 5  51.1    88 sst_5 -1.80           5 1993-01-01  1993     1         12
## 6  51.1    88 sst_6 -1.80           6 1993-04-01  1993     4         15
## # i 4 more variables: month_sin <dbl>, month_cos <dbl>, lon_sin <dbl>,
## #   lon_cos <dbl>
```

Quick Table Checks

Use this section to understand the rows that the machine-learning model will see.

```
cat("Number of rows:", nrow(df), "\n")
```

```
## Number of rows: 131544
```

```
cat("Number of years:", n_distinct(df$year), "\n")
```

```
## Number of years: 29
```

```
cat("Years:", min(df$year), "to", max(df$year), "\n")
```

```
## Years: 1992 to 2020
```

```
cat("Number of grid locations:", n_distinct(paste(df$lat, df$lon)), "\n")
```

```
## Number of grid locations: 1134
```

```
summary(df$sst)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   -1.80   11.02   21.48   18.96   27.17   31.33
```

TODO: Additional Exploration

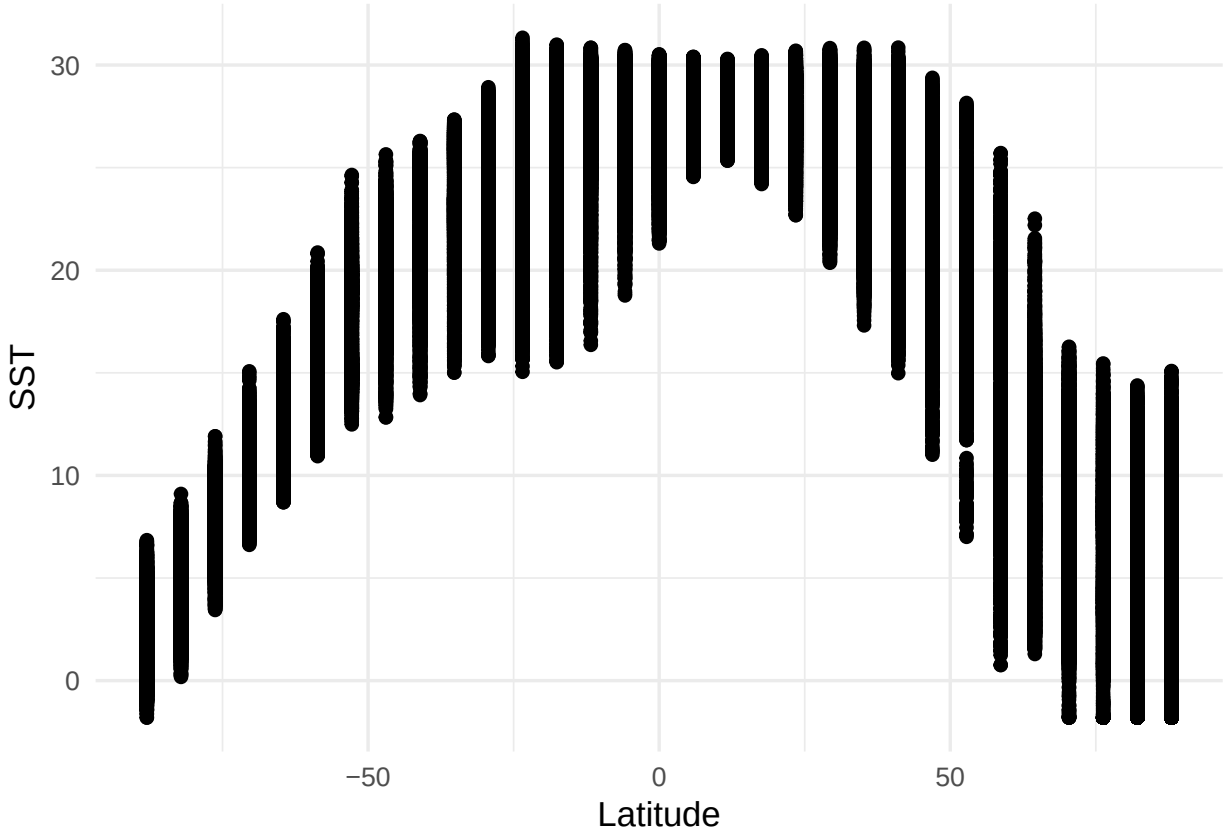
Add at least two exploratory checks of your own.

Examples:

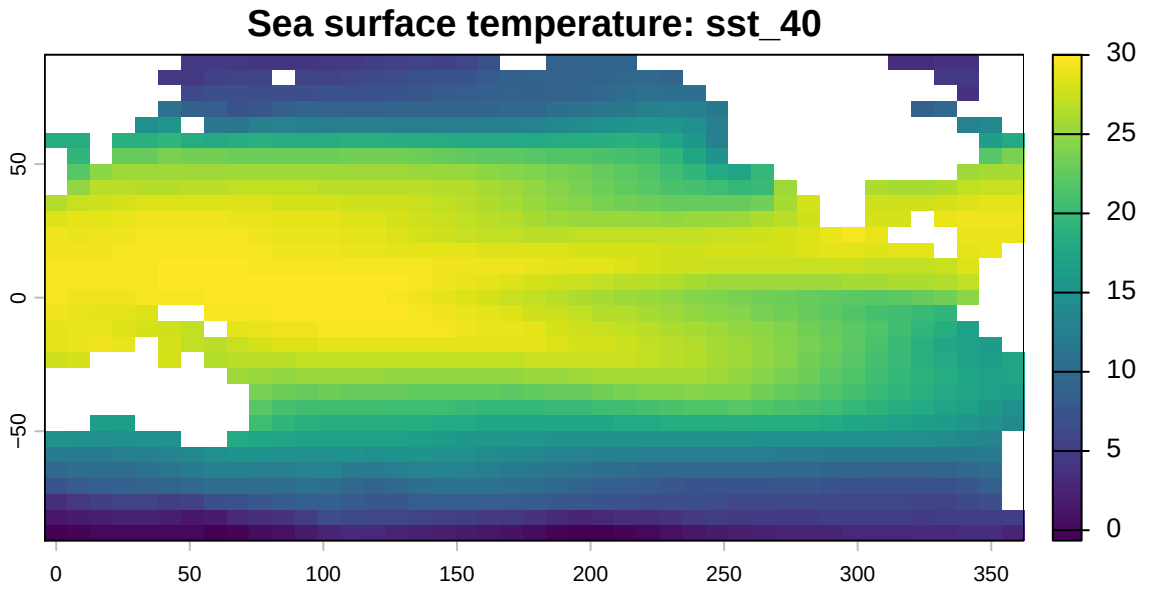
- Plot SST versus latitude.
- Plot SST versus month.
- Compare SST distributions for different years.
- Map a different time step.
- Compute average SST by latitude band.

```
# TODO: Add at least two exploratory checks of your own.

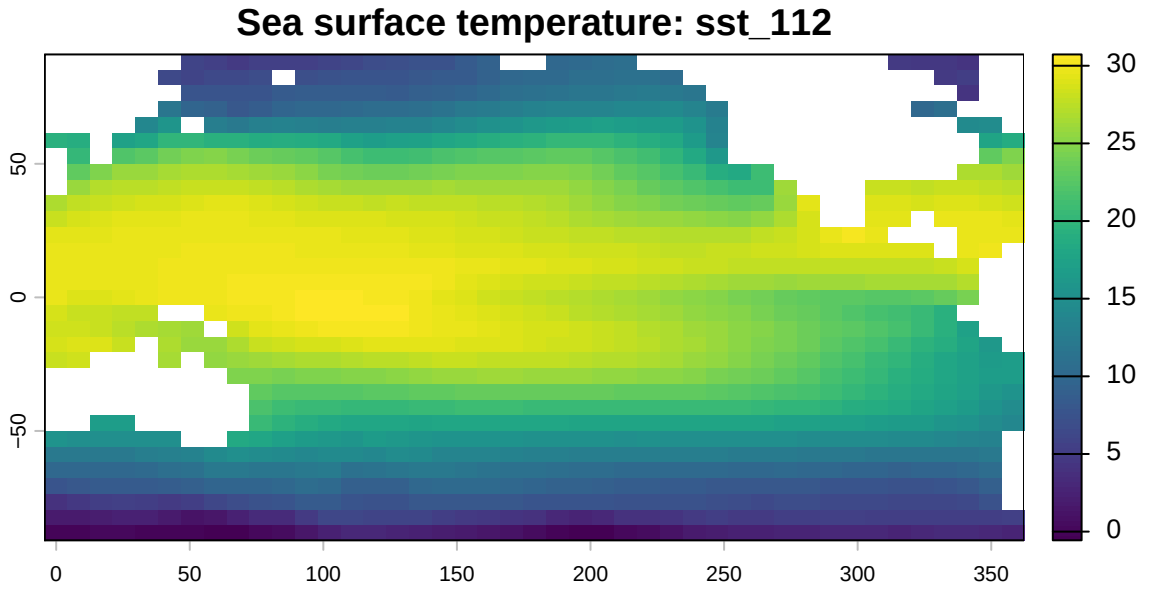
ggplot(df, aes(x = lat, y = sst)) +
  geom_point() +
  labs(
    #title = "Regional mean SST over time",
    x = "Latitude",
    y = "SST"
  ) +
  theme_minimal(base_size = 13)
```




```
oct2001 <- sst_rast[[40]]
plot(oct2001, main = paste("Sea surface temperature:", names(oct2001)))
```

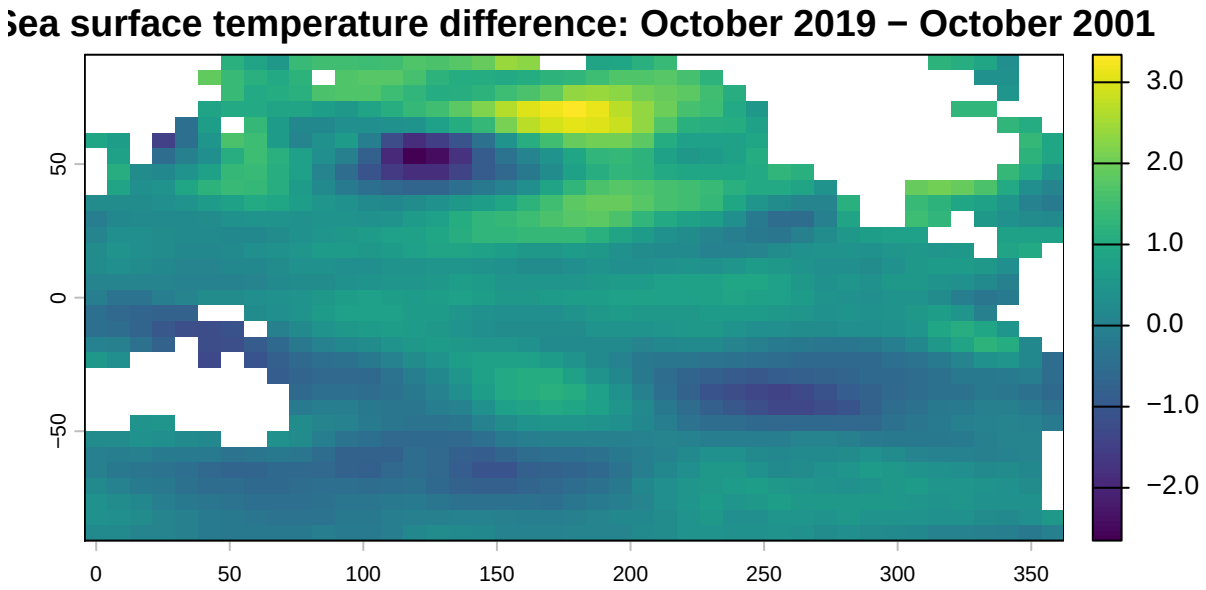



```
oct2019 <- sst_rast[[112]]
plot(oct2019, main = paste("Sea surface temperature:", names(oct2019)))
```




```
oct_diff <- oct2019 - oct2001

plot(oct_diff, main = paste("Sea surface temperature difference: October 2019 - October 2001"))
```



TODO: Historical Monthly SST Feature

Create a historical average SST feature for each location and month. Be careful not to use future data when creating this column.

The idea is:

1. Group by location and month.
2. Sort by time.
3. Compute the cumulative mean from previous observations only.
4. Use `lag()` so the current observation is not used to predict itself.

```
# TODO: Create Month_Average_sst without using future data.
# Hint: use lag(cummean(sst)).

# Example scaffold:
df <- df %>%
   group_by(lat, lon, month) %>%
   mutate(Month_Average_sst = dplyr::lag(cummean(sst))) %>%
   ungroup()

df <- df %>%
  drop_na(Month_Average_sst)
```

4. Define the Prediction Problem

We will predict SST from simple location and time features. This is not meant to be the best possible ocean forecasting model. The goal is to study how evaluation changes when data are spatially and temporally correlated.

```
feature_cols <- c(
  "lat",
  "lon_sin",
  "lon_cos",
  "month_sin",
  "month_cos",
```

```
  "time_index",
  "Month_Average_sst"
)

target_col <- "sst"

# If Month_Average_sst does not exist yet, stop with a helpful reminder.
stopifnot("Month_Average_sst" %in% names(df))

model_data <- df %>%
  select(all_of(c(target_col, feature_cols, "year", "lat", "lon", "time"))) %>%
  mutate(row_id = row_number())

head(model_data)
```

```
## # A tibble: 6 x 12
##     sst   lat lon_sin lon_cos month_sin month_cos time_index
##   <dbl> <dbl>   <dbl>   <dbl>     <dbl>     <dbl>      <dbl>
## 1 -1.80    88   0.779   0.627     0.5       0.866         12
## 2 -1.80    88   0.779   0.627     0.866    -0.5           15
## 3 11.9     88   0.779   0.627    -0.5      -0.866         18
## 4  6.23    88   0.779   0.627    -0.866     0.5           21
## 5 -1.80    88   0.779   0.627     0.5       0.866         24
## 6 -1.80    88   0.779   0.627     0.866    -0.5           27
## # i 5 more variables: Month_Average_sst <dbl>, year <dbl>, lon <dbl>,
## #   time <date>, row_id <int>
```

5. Create a Reusable Model Workflow

We will use a tree-based regression model. Tree-based models can capture nonlinear patterns such as warmer temperatures near the equator and seasonal variation.

In `tidymodels`, we define:

- a **recipe** for preprocessing,
- a **model specification**, and
- a **workflow** that combines them.

```
sst_recipe <- recipe(sst ~ lat + lon_sin + lon_cos + month_sin + month_cos + time_index + Month_Average_
                     data = model_data) %>%
  step_impute_median(all_numeric_predictors())

rf_spec <- rand_forest(
  trees = 300,
  min_n = 5,
  mtry = 4
) %>%
  set_mode("regression") %>%
  set_engine("ranger", importance = "impurity")

sst_workflow <- workflow() %>%
  add_recipe(sst_recipe) %>%
  add_model(rf_spec)

sst_workflow
```

```
## == Workflow ====================================================================
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor ----------------------------------------------------------------
## 1 Recipe Step
##
## * step_impute_median()
##
## -- Model -----------------------------------------------------------------------
## Random Forest Model Specification (regression)
##
## Main Arguments:
##   mtry = 4
##   trees = 300
##   min_n = 5
##
## Engine-Specific Arguments:
##   importance = impurity
##
## Computational engine: ranger
```

```
summarize_resamples <- function(resample_results, label) {
  collect_metrics(resample_results) %>%
    select(.metric, mean, std_err) %>%
    pivot_wider(
      names_from = .metric,
      values_from = c(mean, std_err)
    ) %>%
    transmute(
      Evaluation = label,
      `Mean RMSE` = mean_rmse,
      `SE RMSE` = std_err_rmse,
      `Mean MAE` = mean_mae,
      `Mean R2` = mean_rsq
    )
}

regression_metrics <- metric_set(rmse, mae, rsq)
```

6. Naive Evaluation: Random k-Fold Cross-Validation

Random k-fold cross-validation mixes observations from different locations and times into each fold. This is common in introductory machine learning, but it is often too optimistic for spatial-temporal data because very similar observations may appear in both training and test folds.

```
random_cv <- vfold_cv(model_data, v = 5)

random_results <- fit_resamples(
  sst_workflow,
  resamples = random_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
```

```
)

collect_metrics(random_results)
```

```
## # A tibble: 3 x 6
##   .metric .estimator  mean     n   std_err .config
##   <chr>   <chr>      <dbl> <int>     <dbl> <chr>
## 1 mae     standard   0.336     5 0.00126   pre0_mod0_post0
## 2 rmse    standard   0.468     5 0.00251   pre0_mod0_post0
## 3 rsq     standard   0.997     5 0.0000303 pre0_mod0_post0
```

```
random_summary <- summarize_resamples(random_results, "Random k-fold CV")
random_summary
```

```
## # A tibble: 1 x 5
##   Evaluation       `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>                  <dbl>     <dbl>      <dbl>     <dbl>
## 1 Random k-fold CV       0.468   0.00251      0.336     0.997
```

Discussion: Random CV

Questions:

1. What is the average RMSE under random k-fold CV?
2. Why might this number be too optimistic?
3. In this dataset, what kinds of information can leak from the training folds into the test fold?

Your answer: The average RMSE under random k-fold CV is 0.468. It could be too optimistic because simlar observations may be in the training and test folds. Autocorrelated information could be leaked, like regional SST or seasonal SST. TODO

7. Temporal Block Cross-Validation

Now we will hold out entire years. This asks a different question:

> Can the model generalize to years it has not seen during training?

This is a better approximation of forecasting or prediction into future time periods than random cross-validation.

```
# TODO: Complete temporal block cross-validation.

# Step 1: create grouped folds by year.
temporal_cv <- group_vfold_cv(model_data,  group = year, v=5) # replace NULL with group_vfold_cv(...)

# Step 2: run blocked cross-validation.
temporal_results <-  fit_resamples(
  sst_workflow,
  resamples = temporal_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
) # replace NULL with fit_resamples(...)

# Step 3: display the results.
temporal_results
```

```
## # Resampling results
```

```
## # Group 5-fold cross-validation
## # A tibble: 5 x 5
##   splits                 id        .metrics          .notes   .predictions
##   <list>                 <chr>     <list>            <list>   <list>
## 1 <split [99792/27216]>  Resample1 <tibble [3 x 4]> <tibble> <tibble>
## 2 <split [99792/27216]>  Resample2 <tibble [3 x 4]> <tibble> <tibble>
## 3 <split [99792/27216]>  Resample3 <tibble [3 x 4]> <tibble> <tibble>
## 4 <split [104328/22680]> Resample4 <tibble [3 x 4]> <tibble> <tibble>
## 5 <split [104328/22680]> Resample5 <tibble [3 x 4]> <tibble> <tibble>
```

TODO: Summarize Temporal CV

Use `summarize_resamples()` to create a one-row summary for temporal block CV.

```
# TODO: Summarize your temporal CV results.
temporal_summary <- summarize_resamples(temporal_results, "Temporal CV")
temporal_summary
```

```
## # A tibble: 1 x 5
##   Evaluation  `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>             <dbl>     <dbl>      <dbl>     <dbl>
## 1 Temporal CV       0.596    0.0236      0.439     0.996
```

Discussion: Temporal Blocking

Questions:

1. Is temporal block CV better or worse than random CV?
2. What does the difference tell you?
3. Does the model appear to generalize well to unseen years?
4. Would this be a fair test if your real goal were to predict future years?

Your answer: The temporal CV has a worse RMSE than the random CV. The difference suggestes the random CV was relying on information from temporal or seasonal autocorrelation. The model does not seem to generalize well to unseen years. I think this is a fair test if the goal is to create predictions of future years. TODO

8. Spatial Block Cross-Validation

Now we will hold out entire regions. This asks:

> Can the model generalize to locations it has not seen during training?

Spatial blocking is useful when your real prediction task involves mapping unsampled locations or transferring a model to a new region.

TODO: Variogram of Residuals

Use cross-validated residuals from the random CV model to estimate the spatial scale of residual autocorrelation. This avoids using purely in-sample residuals, which can understate model error.

Hint: Look back at the earlier spatial regression activity. One reasonable approach is:

1. Fit the workflow once to the full modeling table.
2. Predict SST for the same rows.
3. Convert to overall average observed and modeled sst for each location.
4. Compute residuals.
5. Convert into an `sf`/`sp` object.

6. Use `gstat::variogram()`.

```
# TODO: Create a variogram of model residuals to estimate spatial autocorrelation distance.

random_predictions <- collect_predictions(random_results)

residual_data <- random_predictions %>%
  mutate(residual = sst - .pred) %>%
  left_join(
    model_data %>%
      mutate(.row = row_number()) %>%
      select(.row, lat, lon, time),
    by = ".row"
  )

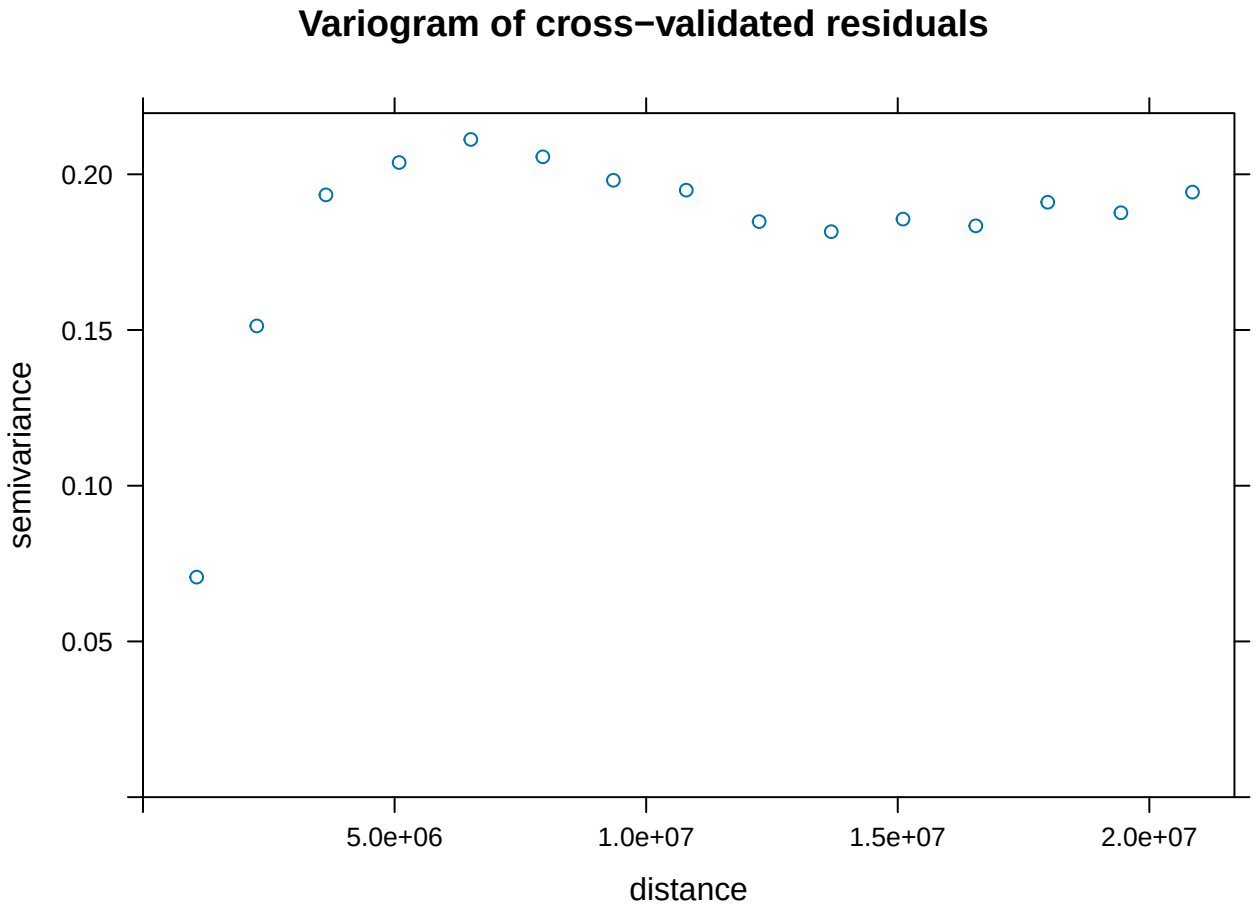
residual_one_time <- residual_data %>%
  filter(time == min(time)) %>%
  sample_n(size = min(3000, n()))

residual_sf <- st_as_sf(
  residual_one_time,
  coords = c("lon", "lat"),
  crs = 4326
) %>%
  st_transform(3857)

residual_sp <- as(residual_sf, "Spatial")

residual_variogram <- variogram(residual ~ 1, data = residual_sp)

plot(residual_variogram, main = "Variogram of cross-validated residuals")
```



Create Spatial Blocks

Once you have estimated the extent of spatial autocorrelation, use that information to set the block size.

```
# TODO: choose block sizes based on your variogram or exploratory reasoning.
lat_block_size <- 5 # replace NULL with a numeric block size, such as 5
lon_block_size <- 5 # replace NULL with a numeric block size, such as 5

model_data <- model_data %>%
  mutate(
    lat_block = floor((lat + 90) / lat_block_size),
    lon_block = floor(lon / lon_block_size),
    spatial_block = paste(lat_block, lon_block, sep = "_")
  )

cat("Number of spatial blocks:", n_distinct(model_data$spatial_block), "\n")
```

```
## Number of spatial blocks: 1134
```

```
model_data %>%
  select(lat, lon, lat_block, lon_block, spatial_block) %>%
  distinct() %>%
  head()
```

```
## # A tibble: 6 x 5
##     lat   lon lat_block lon_block spatial_block
```

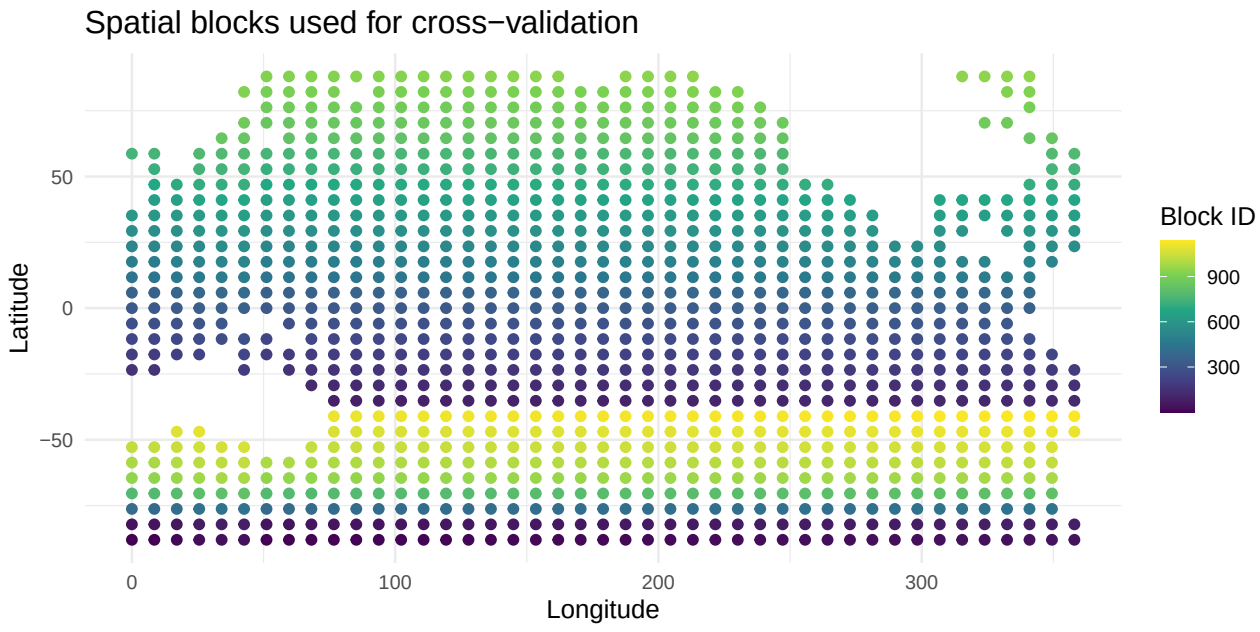
```
##   <dbl> <dbl>     <dbl>     <dbl> <chr>
## 1    88  51.1        35        10 35_10
## 2    88  59.7        35        11 35_11
## 3    88  68.2        35        13 35_13
## 4    88  76.7        35        15 35_15
## 5    88  85.2        35        17 35_17
## 6    88  93.8        35        18 35_18
```

Visualize the Spatial Blocks

Each point below is one grid location. The colors represent block IDs, not physical values.

```
block_map <- model_data %>%
  distinct(lat, lon, spatial_block) %>%
  mutate(block_code = as.integer(factor(spatial_block)))

ggplot(block_map, aes(x = lon, y = lat, color = block_code)) +
  geom_point(size = 2) +
  coord_equal() +
  scale_color_viridis_c() +
  labs(
    title = "Spatial blocks used for cross-validation",
    x = "Longitude",
    y = "Latitude",
    color = "Block ID"
  ) +
  theme_minimal(base_size = 13)
```



Coding Task: Spatial Block CV

Complete the spatial block cross-validation.

Hints:

- Use `group_vfold_cv()` again.
- The grouping variable should be `spatial_block`.

- The block columns are used only to define folds; they are not predictors in the recipe.

```
# TODO: Complete spatial block cross-validation.

# Step 1: create grouped folds by spatial block.
spatial_cv <-  group_vfold_cv(model_data,  group = spatial_block, v=5) # replace NULL with group_vfold_

# Step 2: run blocked cross-validation.
spatial_results <- fit_resamples(
  sst_workflow,
  resamples = spatial_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
) # replace NULL with fit_resamples(...)

# Step 3: display the results.
spatial_results
```

```
## # Resampling results
## # Group 5-fold cross-validation
## # A tibble: 5 x 5
##   splits                 id        .metrics         .notes   .predictions
##   <list>                 <chr>     <list>           <list>   <list>
## 1 <split [101584/25424]> Resample1 <tibble [3 x 4]> <tibble> <tibble>
## 2 <split [101584/25424]> Resample2 <tibble [3 x 4]> <tibble> <tibble>
## 3 <split [101584/25424]> Resample3 <tibble [3 x 4]> <tibble> <tibble>
## 4 <split [101584/25424]> Resample4 <tibble [3 x 4]> <tibble> <tibble>
## 5 <split [101696/25312]> Resample5 <tibble [3 x 4]> <tibble> <tibble>
```

Coding Task: Summarize Spatial CV

Use `summarize_resamples()` to create a one-row summary for spatial block CV.

```
# TODO: Summarize your spatial CV results.
spatial_summary <-summarize_resamples(spatial_results, "Spatial summary")

spatial_summary
```

```
## # A tibble: 1 x 5
##   Evaluation      `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>                 <dbl>     <dbl>      <dbl>     <dbl>
## 1 Spatial summary       0.460   0.00645      0.329     0.997
```

Discussion: Spatial Blocking

Questions:

1. Is spatial block CV better or worse than random CV?
2. What does this say about the model's ability to predict in new locations?
3. Why is latitude likely such a powerful predictor of SST?
4. Why might a model still perform well in spatial blocks if the region has a strong latitudinal gradient?

Your answer: Spatial block is about the same as the random CV. It does okay predicitng in new locations. Latitude is strongly linked ot increased temperatures and is where the two hemispheres gyres meet creating the Equatorial current. These gyres have and temperature associated with latitude have a large impact on SST. So blocks will likey not blind the model to latitudinal trends, and there is still seasonal trends within blocks. TODO

9. Spatial-Temporal Holdout

Spatial blocking asks whether the model can generalize to new places. Temporal blocking asks whether the model can generalize to new years. A spatial-temporal holdout combines both ideas.

Here we create a stricter test: the test data must come from held-out spatial blocks **and** held-out years. The training data exclude those spatial blocks and those years.

```
make_strict_spatiotemporal_splits <- function(data, spatial_col = "spatial_block", time_col = "year", v
  spatial_blocks <- sort(unique(data[[spatial_col]]))
  time_blocks <- sort(unique(data[[time_col]]))

  spatial_fold_ids <- sample(rep(seq_len(v), length.out = length(spatial_blocks)))
  time_fold_ids <- sample(rep(seq_len(v), length.out = length(time_blocks)))

  spatial_lookup <- tibble(
    spatial_block_value = spatial_blocks,
    spatial_fold = spatial_fold_ids
  )

  time_lookup <- tibble(
    time_block_value = time_blocks,
    time_fold = time_fold_ids
  )

  split_list <- vector("list", v)

  for (fold_id in seq_len(v)) {
    heldout_spatial <- spatial_lookup %>%
      filter(spatial_fold == fold_id) %>%
      pull(spatial_block_value)

    heldout_time <- time_lookup %>%
      filter(time_fold == fold_id) %>%
      pull(time_block_value)

    assessment_idx <- which(data[[spatial_col]] %in% heldout_spatial & data[[time_col]] %in% heldout_ti
    analysis_idx <- which(!(data[[spatial_col]] %in% heldout_spatial) & !(data[[time_col]] %in% heldout_

    split_list[[fold_id]] <- make_splits(
      list(analysis = analysis_idx, assessment = assessment_idx),
      data = data
    )
  }

  manual_rset(
    splits = split_list,
    ids = paste0("Fold", seq_len(v))
  )
}
```

```
strict_st_cv <- make_strict_spatiotemporal_splits(
  model_data,
  spatial_col = "spatial_block",
  time_col = "year",
```

```
  v = 5
)

strict_st_results <- fit_resamples(
  sst_workflow,
  resamples = strict_st_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
)

collect_metrics(strict_st_results)
```

```
## # A tibble: 3 x 6
##   .metric .estimator  mean     n  std_err .config
##   <chr>   <chr>      <dbl> <int>    <dbl> <chr>
## 1 mae     standard   0.441     5 0.0126   pre0_mod0_post0
## 2 rmse    standard   0.597     5 0.0189   pre0_mod0_post0
## 3 rsq     standard   0.996     5 0.000369 pre0_mod0_post0
```

```
strict_st_summary <- summarize_resamples(strict_st_results, "Spatial-temporal block CV")
strict_st_summary
```

```
## # A tibble: 1 x 5
##   Evaluation                `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>                           <dbl>     <dbl>      <dbl>     <dbl>
## 1 Spatial-temporal block CV       0.597    0.0189      0.441     0.996
```

Discussion: Spatial-Temporal Holdout

The spatial-temporal holdout is the hardest evaluation in this homework because the model is asked to predict observations from a region and years that were excluded from training.

If this error is much larger than the random CV error, that is not a failure of the model. It is evidence that the original random CV estimate was answering an easier question.

10. Compare All Evaluation Strategies

After you complete the temporal, spatial, and spatial-temporal sections, combine the summary tables.

```
# Combine all summaries into one table.
comparison <- bind_rows(
  random_summary,
  temporal_summary,
  spatial_summary,
  strict_st_summary
)

comparison
```

```
## # A tibble: 4 x 5
##   Evaluation                `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>                           <dbl>     <dbl>      <dbl>     <dbl>
## 1 Random k-fold CV                0.468   0.00251      0.336     0.997
## 2 Temporal CV                     0.596   0.0236       0.439     0.996
## 3 Spatial summary                 0.460   0.00645      0.329     0.997
```

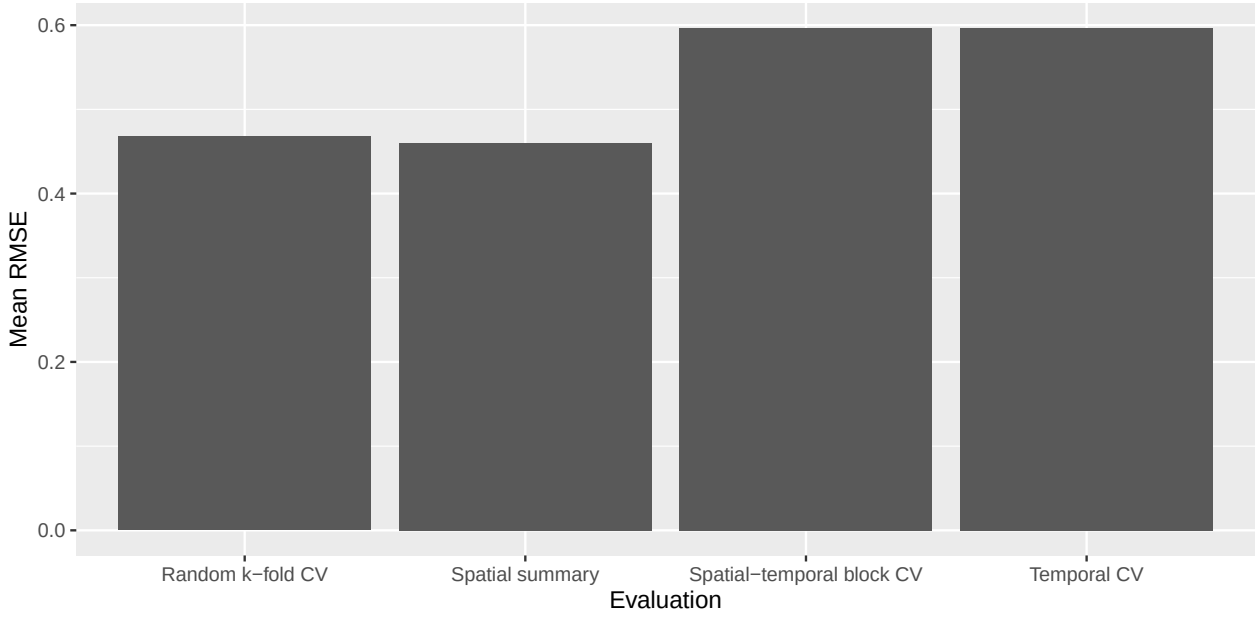
```
## 4 Spatial-temporal block CV       0.597   0.0189       0.441     0.996
```

Plot the Comparison

Complete the plotting code below. You should produce a bar chart of mean RMSE for the evaluation strategies.

```
# TODO: Create a bar chart of Mean RMSE by Evaluation strategy.
# Hint: use

ggplot(comparison, aes(x = Evaluation, y = `Mean RMSE`)) + geom_col()
```



11. Final Reflection

Write a brief paragraph answering the following:

1. Which evaluation strategy gave the most optimistic result?
2. Which evaluation strategy gave the most conservative result?
3. Which strategy best matches a forecasting problem?
4. Which strategy best matches prediction at unsampled locations?
5. Why should environmental scientists be cautious when reporting random cross-validation accuracy for spatial or temporal datasets?

Your answer: The Random k-fold CV and spatial blocking gave the most optimisitc result. The most conservative results were from the temporal and spatial-temporal blocks. The spatial-temporal block best matches a forecasting problem. Spatial block would best match prediction at unsampled locations. Scientists reporting random cross validation accuracy could have data leakage from spatial and temporal datasets that would create over optimistic results. TODO